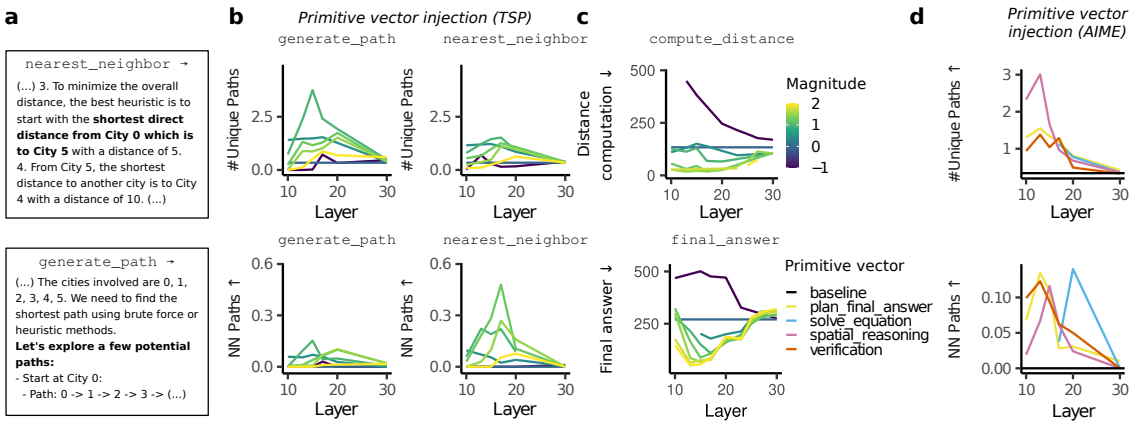

Figure 3: Primitive vector injection induces associated algorithmic behavior. Injecting the `nearest_neighbor` or `generate_path` primitive vectors directly after the prompt **a** increases expression in output (see examples). **b** Varying the injection layer and the magnitude modifies the number of unique paths generated (top row) and the proportion of nearest-neighbor paths (bottom row). **c** Injecting primitive vectors for `compute_distance` in the middle of the reasoning trace increases relevant behavioral hallmarks in the output. **d** Primitive vectors from AIME were injected to different layers while solving Traveling Salesperson (TSP), showing cross-task primitive transfer and algorithmic induction.

to more quickly compute the distance of a candidate path. Interestingly, the reverse is also true: subtracting the primitive makes the model *less* likely to implement a distance computation (Fig. 3c). This suggests that subtracting primitive vectors can prevent associated algorithmic primitives. More broadly, we evaluate six behavioral hallmarks across six possible function vectors and find that all behavioral hallmarks are most strongly induced by a candidate primitive vector with a relevant computational role (Table 2; see further control experiments in App. F.1, F.3).

Notably, the role of a particular cluster cannot always be inferred from the tokens on which it is active alone; the surrounding context often also plays a role. For example, the cluster `compare_and_verify` is largely active on the token corresponding to the final distance of a candidate path. However, we noticed that tokens on which this cluster was active were often followed by subsequent checks and comparisons. We therefore hypothesized that this cluster may not just represent the final distance, but also induce a verification primitive. Table 2 confirms this hypothesis. Interestingly, we can observe this dual nature in representational space as well. In a t-SNE plot of the average representational trajectory of these example primitives, we observe that in earlier layers, `compare_or_verify` evolves along other path-related primitives like `generate_new_path`, whereas it moves closer to `compare_or_verify2` in later layers (Fig. 2b).

Comparing Algorithmic Primitives between Models. To compare primitives between Phi-4 and Phi-4-Reasoning, we computed the dissimilarity between the primitive frequencies involved in their responses. We found that responses by Phi-4-Reasoning are highly stereotyped (Fig. 4a). In contrast, Phi-4 has two subgroups of distinct responses. Notably, these groups map onto two distinct approaches towards solving TSP: a brute-force search and a random guess without further refinement. This highlights that algorithmic fingerprinting reveals both differences between models, and differences between responses from the same model. Beyond TSP, the responses from Phi-4 and Phi-4-Reasoning on 3-SAT and GraphNav are highly stereotyped (Fig. 4b,d). On AIME, algorithmic primitives involved in responses to the same question were very similar to each other, whereas primitives involved in responses to different questions were substantially more different, reflecting the higher diversity of questions in AIME (Fig. 4c).

Next, we analyzed which primitives are more common in Phi-4 or Phi-4-Reasoning by computing the signed χ-squared difference, and analyzing the clusters with the largest positive and negative values. This allows us to identify important differences in response strategies across the two models. For example, on TSP and 3-SAT, distinct heuristic approaches arise more commonly in Phi-4-Reasoning than Phi-4 (`nearest_neighbor` and `if_clause_true`, App. A). Beyond task-specific strategies, our comparative analysis also highlights broader differences between the two models: in particular, clusters that are more common in Phi-4-Reasoning relate to general-purpose reasoning steps such as verification, recalling instructions, or planning the final answer.

Compositional Primitive Induction. So far, we have considered sequential compositions of algorithmic primitives. To investigate arithmetic compositions of primitives, we consider a set of algorithmic in-context learning tasks that require identifying 1) the last node of a path ("Terminal node recognition," TNR), 2) the node with the higher reward

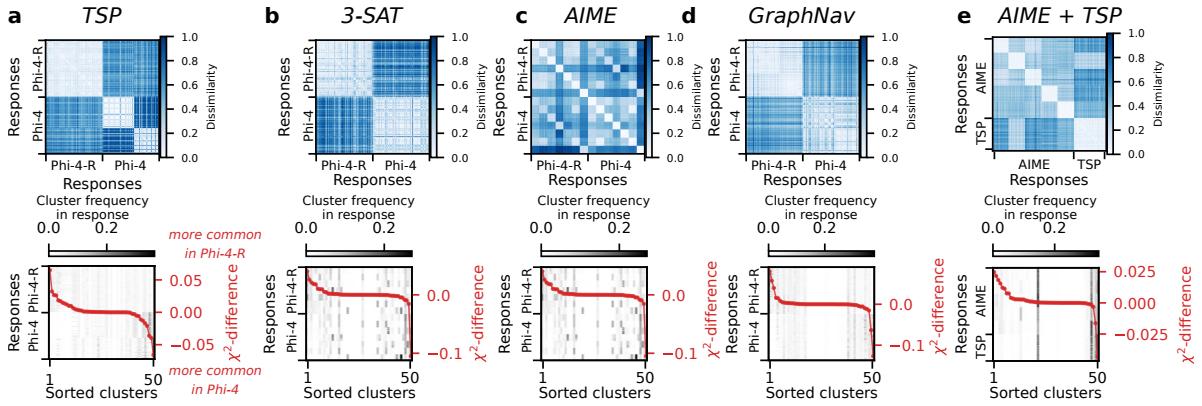

Figure 4: Primitive Cluster Patterns in Phi-4 and Phi-4-Reasoning. **a-d** *top* Normalized dissimilarity of primitive cluster frequencies between Phi-4 and Phi-4-Reasoning for a) TSP, b) 3-SAT, c) AIME, and d) GraphNav. *bottom* Clusters sorted by whether they appear more frequently in Phi-4-Reasoning responses (positive difference) or in Phi-4 responses (negative difference). The lineplot specifies the differences whereas the rasterplot underneath specifies the relative frequencies of the different clusters per response. **e** *left* Dissimilarity of primitive cluster frequencies between Phi-4-Reasoning responses to AIME and TSP. *right* Clusters sorted by whether they occur more frequently in AIME or TSP. The rasterplot underneath again specifies the relative frequencies of the different clusters per response.

("Reward comparison," RC; reward was defined by a number between 1 and 100); and 3) the most highly rewarded node between two paths, a composition of TNR and RC (see example prompt in App. E.1). Remarkably, we find that adding together the primitive vectors for TNR and RC induces this composite behavior in Llama-3-8B, causing it to match few-shot performance in a zero-shot setting (Fig. 5a). These effects were more attenuated in Phi-4 and Phi-4-Reasoning, highlighting that different models may require different modes of composition (Fig. 10).

Cross-task and Compositional Generalization. Generalized algorithmic primitives which can be applied across different tasks, can help us understand why reasoning-finetuned models like Phi-4-Reasoning show improvements on tasks they were not finetuned on. To better understand the degree to which algorithmic primitives in Phi-4-Reasoning are shared between tasks, we apply our algorithmic tracing framework to a set of responses from both TSP and AIME (Fig. 4e). This analysis highlights that while there are different algorithmic primitives at play in either task, the tasks also have many primitives in common.

Next, we tested whether cross-task injection of algorithmic primitives would induce behavioral changes. We extracted primitive vectors from an algorithmic in-context learning task that required extracting a path's first or last node (Fig. 5b) and injected them into complex reasoning traces solving TSP immediately after the model had mentioned a particular path. Despite the extremely different domain, injecting these vectors indeed had the expected effect: injecting the `get_first_node` (resp. `get_last_node`) primitive vector caused the model to mention the first node (resp. last node) of the path more often (Fig. 5c).

Finally, we considered relevant primitive vectors from AIME (e.g. `spatial_reasoning`, `plan_final_answer`) and injected them into Phi-4 generating a response to a TSP prompt. We found that these primitive vectors caused the model to generate more paths and a higher proportion of nearest-neighbor paths (Fig. 3d) — a hallmark of Phi-4-Reasoning responses.

6 Discussion

Algorithmic Tracing and Steering. We introduce a framework for tracing and steering algorithmic primitives as building blocks of LLM reasoning with geometric compositionality. We adapted function vector techniques (Todd et al., 2024) to extract minimal, 1-step primitive vectors from carefully designed simple tasks as well as benchmarks with more complex reasoning contexts. When "injected" to LLM layers after the prompt or in the middle of reasoning traces, these primitive vectors increase the behavioral expression of corresponding reasoning steps (Figs. 3 and 5). This extends function vector research beyond simple input-output relations. A central contribution of our work is the compositional exploration of primitive vector arithmetic. Our framework bridges a principled path from laboratory-identified primitives to real-world reasoning behaviors.

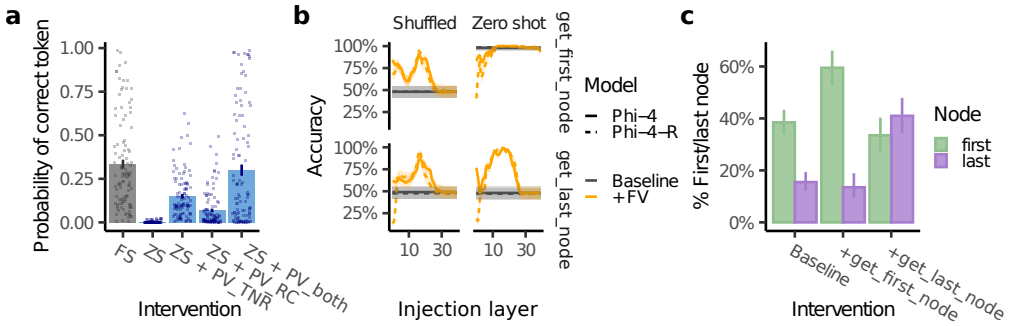

Figure 5: Algorithmic primitives operating over graphs, compositional induction, and cross-task transfer. **a** Injecting the sum of a terminal node recognition primitive vector (PV_TNR) and a reward comparison primitive vector (PV_RC) in a zero-shot setting (ZS) improves model performance on a task requiring a composition of both primitives and recovers few-shot performance (FS). See Fig. 10 for additional models. **b** Primitive vectors for extracting the first or last node of a presented graph reliably improves performance across different injection layers and across a shuffled few-shot and a zero shot-setting. **c** After injecting these primitive vectors into complex reasoning traces solving TSP, the model becomes more likely to mention the corresponding node next.

Toward a Geometry of Compositional Abstraction in LLMs. We find that algorithmic primitives exhibit geometric regularities through compositional operations like addition, subtraction, multiplication, and scalar modulation. The layer and magnitude of injection shape the expression of the primitive (Fig. 3). We show cross-task transfer and generalizability of primitive vectors: spatial reasoning and verification primitives extracted from AIME successfully transfer to TSP. This transferability hints at potentially universal algorithmic building blocks underlying diverse reasoning capabilities. If this is the case, an algorithm may be expressed by an LLM in terms of an algebraic geometry of primitive vectors and their composition.

Reasoning vs. Memorization. Our work contributes to recent discussions about the extent to which LLMs exhibit abstract reasoning rather than relying on memorized procedures, failing to generalize in counterfactual scenarios (Wu et al., 2024; Power et al., 2022; Zhang et al., 2024; Wang et al., 2025). While models were not directly trained on our tasks (Abdin et al., 2025), it is possible that they may be using amortized reasoning based on similar examples in the training. LLMs exhibit better problem-solving in high probability than low probability settings (e.g. McCoy et al., 2024a), which rely on statistical regularities rather than abstract reasoning. Moreover, the absence of metacognitive abilities in LLMs results in brittle problem-solving (Johnson et al., 2024; Lewis & Mitchell, 2024). However, models finetuned for reasoning exhibit less sensitivity to task probability than base models McCoy et al. (2024b) and improved metacognitive-like uncertainty management strategies (e.g., verifying whether a candidate solution is correct) (Guo et al., 2025; Gandhi et al., 2025) that in turn causally improve reasoning (Bogdan et al., 2025). In line with these findings, our analysis identified algorithmic primitives for managing uncertainty (e.g., verification) and in-context recall. Such primitives and their associated behaviors may underlie a shift from memorization toward compositional problem-solving in reasoning models.

Limitations. One limitation is that the primitives, corresponding clusters, and meta-clusters we identify do not always map to a known algorithm. Future work can identify **primitive ontology** and algorithmic logic of different models, identifying commonalities and differences across them. Here, we focus on reasoning as multi-step problem-solving in limited tasks, which may not capture the complexity and multi-modal nature of natural reasoning, studied in cognitive sciences for decades Tversky (2005); Shepard & Metzler (1971); MacGregor & Ormerod (1996). Finally, we have focused on linear compositions of primitive vectors, leaving more complex interactions, e.g. non-linear combinations of several vectors on manifolds, for future work.

Future Directions. The algorithmic tracing and steering framework can be applied to any architecture (e.g., vision, diffusion, and multi-modal models) and domain beyond reasoning. An immediate extension is a more detailed manifold analysis in order to capture the compositional geometry of primitives beyond linear composition, and establish a task-specific and universal **primitive ontology**. Another key direction is the **algorithmic training and finetuning of LLMs**, with algorithmic objectives. Future self-improving models can be designed for **algorithmic self-play**: generating and evaluating compositional algorithmic solutions. Finally, collecting human reasoning data on the same tasks enables us to compare and finetune model-human **algorithmic alignment**.

Conclusion. The identification of universal primitives and their compositional geometry opens new avenues for interpretability research and suggests principled approaches for computational models of human reasoning, model-human alignment, and enhancing LLM reasoning capabilities.